

Integración con IA

Cómo el sistema usa LLMs para generar contenido pedagógico.

Lo esencial

- No entrenamos ni hicimos fine-tuning. Usamos un LLM pre-entrenado vía API.
- Dos proveedores soportados:** Groq (Llama 3.3 70B, default) y Google Gemini. Cambiás con la env var `PLAN_GENERATOR`.
- La estructura del plan (módulos, fases, orden) la decide código determinístico. Solo el contenido textual y los ejercicios los genera el LLM.
- Si el LLM falla, hay fallback automático a `MockPlanGenerator`. El campo `generator_used` en cada plan indica qué se usó (`groq` / `gemini` / `mock`).

Dónde se usa el LLM

Punto	Archivo	Qué pide
Generar units del plan	<code>learning_paths/plan_generator/_base_llm.py</code>	title + content + 5 ejercicios multiple_choice
Sugerir recursos de estudio	<code>resources/suggester.py</code>	videos, guías, sandboxes — cacheados por (skill, fase)
Feedback empático en fallo	<code>attempts/feedback.py</code>	mensaje de Paciencia sin revelar la respuesta

Qué decide el código y qué decide la IA

Decisión	Quién
Skills que entran al plan, orden, prioridad	Código
Cantidad de units por módulo (3), duración (10/15/30 min)	Código
Validación de respuesta del alumno, cálculo de mastery	Código
Title, content y ejercicios de cada unit	LLM
Recursos sugeridos	LLM (con guardrails anti-alucinación de URLs)
Mensaje de feedback ante error	LLM

La estrategia híbrida nos da consistencia (la estructura no varía aunque el LLM se equivoque), permite auditar las reglas de negocio en el código, y reduce costo (menos llamadas).

Proveedores

Aspecto	Groq	Gemini
Modelo usado	llama-3.3-70b-versatile	gemini-2.5-flash-lite
Free tier RPD	14.400	1.500
Latencia típica por call	1-3 s	3-7 s
API key	https://console.groq.com/keys	https://aistudio.google.com/apikey

Para cambiar entre proveedores: editar `PLAN_GENERATOR` en Render → Environment. Save → redeploya solo. No hay que tocar código.

Para cambiar el modelo dentro del mismo proveedor: editar `GROQ_MODEL` o `GEMINI_MODEL`.

Configuración

Variable	Valores
<code>PLAN_GENERATOR</code>	<code>mock</code> / <code>groq</code> / <code>gemini</code>
<code>GROQ_API_KEY</code> , <code>GROQ_MODEL</code>	requeridas si <code>PLAN_GENERATOR=groq</code>
<code>GEMINI_API_KEY</code> , <code>GEMINI_MODEL</code>	requeridas si <code>PLAN_GENERATOR=gemini</code>

Las variables del proveedor inactivo se pueden dejar configuradas — el código las ignora.

Manejo de fallas

Todas las fallas del LLM (key faltante, 429 por cuota, 404 por modelo, JSON inválido) caen al fallback Mock. El plan se persiste igual con `generator_used: "mock"`. Si el suggerter de resources falla, las units quedan con `resources: []`. Si el feedback de attempts falla, se devuelve un texto estático.

La única falla que el sistema no oculta es el timeout de Render (30s en free tier). En ese caso el cliente recibe error HTTP y el plan no se persiste.

Para defensa del TP

No entrenamos ni hicimos fine-tuning. Usamos LLMs pre-entrenados de propósito general (Groq Llama 3.3 70B como default, Gemini como alternativa) y aplicamos prompt engineering para alinear la salida al modelo pedagógico 5P. La inteligencia del dominio vive en nuestros prompts (`prompts.py`) y en la estructura híbrida que mantiene las reglas pedagógicas en código. El LLM aporta capacidad de generación de texto natural y razonamiento general.

Reglas que mantener

1. **No mover lógica de negocio a los prompts.** El prompt define tono y formato; las reglas viven en `_base_llm.py` y `service.py`.
2. **No remover el fallback a Mock.** Es lo que garantiza disponibilidad sin LLM.
3. **No cachear respuestas del LLM por estudiante.** Cada plan es personalizado a sus intereses. La excepción es `resources`, que cachea por (skill, fase) porque es independiente del estudiante.